

به نام خدا

تمرین سری ۱ مدیریت امنیت اطلاعات

امیرحسین قدیری ۴۰۴۲۰۵۰۳۱

بخش ۲.۱

به منظور راه اندازی swtpm از دستورات زیر استفاده می شود.

```
rm -rf /tmp/swtpm
mkdir -p /tmp/swtpm
```

```
swtpm socket --tpm2 --server type=tcp,port=2321 --ctrl type=tcp,port=2322 --tpmstate
dir=/tmp/swtpm --flags startup-clear
```

با استفاده از این دستورات، یک سرور TPM 2.0 بر روی پورت 2321 و یک سرور کنترلی بر روی پورت 2322 راه اندازی می شود. در مسیر /tmp/swtpm نیز وضعیت TPM ذخیره شده است.

```
skye@arch ~/W/u/ce879-042-404205031> rm -rf /tmp/swtpm
skye@arch ~/W/u/ce879-042-404205031> mkdir -p /tmp/swtpm
skye@arch ~/W/u/ce879-042-404205031> swtpm socket --tpm2 --server type=tcp,port=2321 --ctrl type=tcp,port=2322 \
--tpmstate dir=/tmp/swtpm --flags startup-clear
```

با تنظیم مقدار متغیر TPM2TOOLS_TCTI می توان در یک ترمینال دیگر از دستور tpm2_getrandom استفاده نمود.

```
export TPM2TOOLS_TCTI="swtpm:host=localhost,port=2321"
tpm2_getrandom 4 -o out.bin
hexdump out.bin
```

```
skye@arch ~/W/u/ce879-042-404205031> export TPM2TOOLS_TCTI="swtpm:host=localhost,port=2321"
skye@arch ~/W/u/ce879-042-404205031> tpm2_getrandom 4 -o out
skye@arch ~/W/u/ce879-042-404205031> hexdump out
00000000 a0fa 2c87
00000004
skye@arch ~/W/u/ce879-042-404205031>
```

بخش ۲.۲

ابتدا با استفاده از دستور زیر یک Endorsement Key ساخته شده است.

```
tpm2_createek -c ek-1.ctx -u ek-1.pub
```

در ادامه با حذف دایرکتوری /tmp/swtpm و راه اندازی دوباره swtpm، وضعیت TPM دوباره راه اندازی شده و یک Endorsement Key دیگر می سازیم.

```
tpm2_createek -c ek-2.ctx -u ek-2.pub
```

با مقایسه دو کلید ساخته شده، مشاهده می شود که این دو متفاوت می باشند. مقایسه دو کلید ساخته شده به صورت زیر است.

```
skye@arch ~/W/u/ce879-042-404205031> tpm2_createek -c ek-1.ctx -u ek-1.pub
skye@arch ~/W/u/ce879-042-404205031> tpm2_createek -c ek-2.ctx -u ek-2.pub
skye@arch ~/W/u/ce879-042-404205031> diff ek-1.pub ek-2.pub
Binary files ek-1.pub and ek-2.pub differ
skye@arch ~/W/u/ce879-042-404205031 [1]>
```

در ادامه به منظور ساخت یک Attestation Identity Key از دستور زیر استفاده شده است.

```
tpm2_createek -c ek.ctx
```

```
tpm2_createak -C ek.ctx -c aik.ctx -G rsa -g sha256 -s rsassa -u aik.pub -n aik.name
```

```
skye@arch ~/W/u/ce879-042-404205031> tpm2_createek -c ek.ctx
skye@arch ~/W/u/ce879-042-404205031> tpm2_createak -C ek.ctx -c aik.ctx -G rsa -g sha256 -s rsassa -u aik.pub -n aik.name
loaded-key:
  name: 000bf3e94fc7b817bd26207308a9ecc0527a12380f2ff5f68cfc622d09192883ea8f
  qualified name: 000bfd58a91aab588586f13e445ac57550bb28d07c12a20458908f6e1e0aa0859222
skye@arch ~/W/u/ce879-042-404205031>
```

سپس برای persist کردن این کلید از دستورات زیر استفاده شده است. در اینجا، مقدار 0x81010002 یک handle برای دسترسی به این کلید می باشد.

```
tpm2_flushcontext -t
```

```
tpm2_evictcontrol -C o -c aik.ctx 0x81010002
```

```
skye@arch ~/W/u/ce879-042-404205031> tpm2_flushcontext -t
skye@arch ~/W/u/ce879-042-404205031> tpm2_evictcontrol -C o -c aik.ctx 0x81010002
persistent-handle: 0x81010002
action: persisted
skye@arch ~/W/u/ce879-042-404205031>
```

به منظور مشاهده کلیدهای persist شده و مشاهده کلید مورد نظر، از دستورات زیر استفاده شده است.

```
tpm2_getcap handles-persistent
```

```
tpm2_readpublic -c 0x81010002
```

```

● skye@arch ~/W/u/ce879-042-404205031> tpm2_getcap handles-persistent
- 0x81010002
● skye@arch ~/W/u/ce879-042-404205031> tpm2_readpublic -c 0x81010002
name: 000bf3e94fc7b817bd26207308a9ecc0527a12380f2ff5f68cfc622d09192883ea8f
qualified name: 000bfd58a91aab588586f13e445ac57550bb28d07c12a20458908f6e1e0aa0859222
name-alg:
  value: sha256
  raw: 0xb
attributes:
  value: fixedtpm|fixedparent|sensitivedataorigin|userwithauth|restricted|sign
  raw: 0x50072
type:
  value: rsa
  raw: 0x1
exponent: 65537
bits: 2048
scheme:
  value: rsassa
  raw: 0x14
scheme-halg:
  value: sha256
  raw: 0xb
sym-alg:
  value: null
  raw: 0x10
sym-mode:
  value: (null)
  raw: 0x0
sym-keybits: 0
rsa: b12cbf26978b48e9e1be176f9b505f889105dfac9dd3521a52db412218d7190d47d8ab54556e51cc3b00db44701b4e9286c23a3aab29cb2d0685176c80298a915303
4400dc86936a9f352ecbc2187a09631c1db8ad2143ad7627b97519ffee8cab14b0db7a1781646be0bd1daa179bf092e60c0e8c2086bfdec29b8772892b968a80c46bb8a5a
0661a08cbcd3102421cedc2af50062587bf02e07ff54062aa5dd4ee842d845c2abfbb565c412c94a33bd1c9aa652406fe9a00a0cd390c13f489f7b4b0668e27d411c83294
783bdeaa412d5d636f3e6ac8c726aa5c6ce52f009ab17282b6ab1cacff62eb4329cdfa16203e86ebe5b1a69d34b4934564ab275a63
● skye@arch ~/W/u/ce879-042-404205031>

```

برای این بخش ابتدا سه فایل زیر ساخته شده‌اند که به ترتیب مقادیر `FLAG{S4LAB}` و `STATE_HEALTHY` و `STATE_MALICIOUS` در آن‌ها ذخیره شده‌اند.

```
echo -n "FLAG{S4LAB}" > secret.txt
echo -n "STATE_HEALTHY" > healthy.txt
echo -n "STATE_MALICIOUS" > malicious.txt
```

در ادامه وضعیت PCR16 با هش sha256 مقدار `STATE_HEALTHY` گسترش داده شده و یک policy بر اساس مقدار جدید آن ساخته شده است.

```
tpm2_pcrextend 16:sha256=$(sha256sum healthy.txt | cut -d' ' -f1)
tpm2_createpolicy --policy-pcr -l sha256:16 -L policy.dat
```

```
• skye@arch ~/W/u/ce879-042-404205031> tpm2_pcrextend 16:sha256=$(sha256sum healthy.txt | cut -d' ' -f1)
• skye@arch ~/W/u/ce879-042-404205031> tpm2_createpolicy --policy-pcr -l sha256:16 -L policy.dat
  1bd2ebe11f3237acc8748c0616bd32b96718d6d7b3363cfd8d05e95922f36b80
• skye@arch ~/W/u/ce879-042-404205031> █
```

حالا مقدار مخفی ذخیره شده در فایل با استفاده از دستورات زیر seal شده و یک context برای دسترسی به آن ساخته می‌شود.

```
tpm2_createprimary -C o -c primary.ctx
tpm2_create -C primary.ctx -L policy.dat -i secret.txt -u seal.pub -r seal.priv
tpm2_flushcontext -t
tpm2_load -C primary.ctx -u seal.pub -r seal.priv -c seal.ctx
```

```
• skye@arch ~/W/u/ce879-042-404205031> tpm2_createprimary -C o -c primary.ctx
name-alg:
  value: sha256
  raw: 0xb
attributes:
  value: fixedtpm|fixedparent|sensitivedataorigin|userwithauth|restricted|decrypt
  raw: 0x30072
type:
  value: rsa
  raw: 0x1
exponent: 65537
bits: 2048
scheme:
  value: null
  raw: 0x10
scheme-halg:
  value: (null)
  raw: 0x0
sym-alg:
  value: aes
  raw: 0x6
sym-mode:
  value: cfb
  raw: 0x43
sym-keybits: 128
rsa: ba426f1ef67bd46f181d5f4cfe9049a4d6ba0b39542e8f5b45fd8d82af8b7f90634880c115e9df1b9a2ee808e3e1c39208e3fb5ac36134525e305000916ef78e8005
0e5a174c22c3f2abeb58e435a4673756b8bde286a7b1f85137689f6013543708013469fee7f8e5e4ff2834819f9e7df986b5e53fcf9934c281caca6441fc04d5b152aa015
7cf25e2b0025701baef6066237ebb0ddf8f778c184143551ed205727ff2e1f6674ea7f820fa64d1cb52c383dfc2c44929d4b65d21d7eeb1527ad0d73d0243e27ef8b2cf0e
5213119e8a8b0019d3c43785641521636ea45c2444314c069a4ef3ac0a813dc6b458e97469015ff36cdcbfff17bcbf0431434cab5
```

```

• skye@arch ~/W/u/ce879-042-404205031> tpm2_create -C primary.ctx -L policy.dat -i secret.txt -u seal.pub -r seal.priv
name-alg:
  value: sha256
  raw: 0xb
attributes:
  value: fixedtpm|fixedparent
  raw: 0x12
type:
  value: keyedhash
  raw: 0x8
algorithm:
  value: null
  raw: 0x10
keyedhash: 6999cafd64f566e970ad715302eddc0e8cabfb42e3f5cb7937e57405c7becee9
authorization policy: 1bd2ebe11f3237acc8748c0616bd32b96718d6d7b3363cfd8d05e95922f36b80
• skye@arch ~/W/u/ce879-042-404205031> tpm2_flushcontext -t
• skye@arch ~/W/u/ce879-042-404205031> tpm2_load -C primary.ctx -u seal.pub -r seal.priv -c seal.ctx
  name: 000bd6c0ae2a8b0d56c4e724e54aba939d1ee2977c8e4a0a1b32281a203dd77fa8f8
• skye@arch ~/W/u/ce879-042-404205031>

```

حالت اول

در این حالت، بررسی می‌شود که بتوان مقدار مخفی را با موفقیت unseal کرده و آن را مشاهده نمود. این عمل با استفاده از دستورات زیر انجام شده است.

```
tpm2_flushcontext -t
```

```
tpm2_startauthsession --policy-session -S session.ctx
```

```
tpm2_policypcr -S session.ctx -l sha256:16
```

```
tpm2_unseal -c seal.ctx -p "session:session.ctx"
```

```

• skye@arch ~/W/u/ce879-042-404205031> tpm2_flushcontext -t
• skye@arch ~/W/u/ce879-042-404205031> tpm2_startauthsession --policy-session -S session.ctx
• skye@arch ~/W/u/ce879-042-404205031> tpm2_policypcr -S session.ctx -l sha256:16
1bd2ebe11f3237acc8748c0616bd32b96718d6d7b3363cfd8d05e95922f36b80
• skye@arch ~/W/u/ce879-042-404205031> tpm2_unseal -c seal.ctx -p "session:session.ctx"
FLAG{S4LAB}e
• skye@arch ~/W/u/ce879-042-404205031>

```

حالت دوم

در این حالت، مشاهده می‌شود که با راه‌اندازی مجدد وضعیت PCR16 و گسترش آن با استفاده هش sha256 مقدار STATE_MALICIOUS، دیگر امکان unseal کردن مقدار مخفی وجود نداشته و policy نقض می‌شود. این عمل با استفاده از دستورات زیر انجام شده است.

```
tpm2_flushcontext -t
```

```
tpm2_pcrreset 16
```

```
tpm2_pcrextend 16:sha256=$(sha256sum malicious.txt | cut -d' ' -f1)
```

```
tpm2_startauthsession --policy-session -S session.ctx
```

```
tpm2_policypcr -S session.ctx -l sha256:16
```

```
tpm2_unseal -c seal.ctx -p "session:session.ctx"
```

```

● skye@arch ~/W/u/ce879-042-404205031> tpm2_flushcontext -t
● skye@arch ~/W/u/ce879-042-404205031> tpm2_pcrreset 16
● skye@arch ~/W/u/ce879-042-404205031> tpm2_pcrextend 16:sha256=$(sha256sum malicious.txt | cut -d' ' -f1)
○ skye@arch ~/W/u/ce879-042-404205031>
● skye@arch ~/W/u/ce879-042-404205031> tpm2_startauthsession --policy-session -S session.ctx
● skye@arch ~/W/u/ce879-042-404205031> tpm2_policyPCR -S session.ctx -l sha256:16
b8e25358d0f374bb2d9bd43226fdb26961bcd1e3e80cc8e5c1f81993a072cfbf
○ skye@arch ~/W/u/ce879-042-404205031> tpm2_unseal -c seal.ctx -p "session:session.ctx"
WARNING:esys:src/tss2-esys/api/Esys_Unseal.c:295:Esys_Unseal_Finish() Received TPM Error
ERROR:esys:src/tss2-esys/api/Esys_Unseal.c:98:Esys_Unseal() Esys Finish ErrorCode (0x0000099d)
ERROR: Esys_Unseal(0x99d) - tpm:session(1):a policy check failed
ERROR: Unable to run tpm2_unseal
○ skye@arch ~/W/u/ce879-042-404205031 [1]>

```

حالت سوم

در این حالت، مشاهده می‌شود که با راه اندازی مجدد وضعیت PCR16 و گسترش آن با استفاده هش sha256 مقدار STATE_HEALTHY می‌توان مقدار مخفی را با موفقیت unseal کرده و آن را مشاهده نمود. این عمل با استفاده از دستورات زیر انجام شده است.

```

tpm2_flushcontext -t
tpm2_pcrreset 16
tpm2_pcrextend 16:sha256=$(sha256sum healthy.txt | cut -d' ' -f1)

tpm2_startauthsession --policy-session -S session.ctx
tpm2_policyPCR -S session.ctx -l sha256:16
tpm2_unseal -c seal.ctx -p "session:session.ctx"

```

```

● skye@arch ~/W/u/ce879-042-404205031> tpm2_flushcontext -t
● skye@arch ~/W/u/ce879-042-404205031> tpm2_pcrreset 16
● skye@arch ~/W/u/ce879-042-404205031> tpm2_pcrextend 16:sha256=$(sha256sum healthy.txt | cut -d' ' -f1)
○ skye@arch ~/W/u/ce879-042-404205031>
● skye@arch ~/W/u/ce879-042-404205031> tpm2_startauthsession --policy-session -S session.ctx
● skye@arch ~/W/u/ce879-042-404205031> tpm2_policyPCR -S session.ctx -l sha256:16
1bd2ebe11f3237acc8748c0616bd32b96718d6d7b3363cfd8d05e95922f36b80
● skye@arch ~/W/u/ce879-042-404205031> tpm2_unseal -c seal.ctx -p "session:session.ctx"
FLAG{S4LAB}e
○ skye@arch ~/W/u/ce879-042-404205031>

```

سوال ۱

- Platform Hierarchy: این سلسله مراتب تخت کنترل سازنده پلتفرم یا firmware قرار دارد. هدف این سلسله مراتب، مدیریتات تنظیمات سطح پایین TPM، ایجاد کلیدهایی برای استفاده داخلی سیستم و مدیریت برخی قابلیت‌ها در هنگام بوت می‌باشد.
 - Storage Hierarchy: مهم‌ترین سلسله مراتب برای کاربران و برنامه‌ها بوده و برای حفاظت و ذخیره‌سازی کلیدها از آن استفاده می‌شود. هدف این سلسله مراتب ایجاد کلیدهای فضای ذخیره‌سازی و رمزنگاری و محافظت از کلیدهای دیگر می‌باشد.
 - Endorsement Hierarchy: از این سلسله مراتب برای اثبات هویت و اعتبار TPM استفاده می‌شود. هدف این سلسله مراتب نگهداری از Endorsement Key ها، پشتیبانی از Remote Attestation و اثبات معتبر بودن TPM می‌باشد.
 - Null Hierarchy: این سلسله مراتب موقت و بدون ذخیره‌سازی دائمی می‌باشد. هدف این سلسله مراتب ایجاد کلیدها و اشیای موقت و جلوگیری از باقی ماندن داده‌های حساس پس از راه اندازی مجدد می‌باشد.
- در TPM 1.2 تقریباً همه عملکردها به یک سلسله مراتب ذخیره‌سازی واحد وابسته بودند. این موضوع مشکلاتی مانند تمرکز بیش از حد اعتماد روی یک بخش دشواری تفکیک وظایف مدیریتی و محدودیت کنترل دسترسی را ایجاد می‌کرد. اما در TPM 2.0 هر سلسله مراتب مستقل بوده و هر کدام دارای seed و authorization جداگانه می‌باشند. همچنین امکان جداسازی نقش‌ها فراهم شده و مدیریت کلیدها انعطاف‌پذیرتر و ایمن‌تر می‌باشد. به طور مثال اگر کلیدهای Storage Hierarchy به خطر بیافتند، کلیدهای Endorsement Hierarchy همچنان امن باقی خواهند ماند.

سوال ۲

PCR ها در TPM نقش بسیار مهمی در تضمین یکپارچگی فرآیند بوت و همچنین پیاده سازی Remote Attestation دارند. این رجیسترها مقادیر هش مربوط به اجزای مختلف سیستم را در طول بوت ذخیره می کنند تا TPM بتواند تشخیص دهد آیا سیستم در وضعیت قابل اعتماد اجرا شده است یا خیر.

در فرآیند بوت، هر مؤلفه قبل از اجرا اندازه گیری شده و هش می شود. برای مثال هش BIOS یا UEFI ابتدا محاسبه شده و نتیجه در یکی از PCR ها ثبت می گردد. سپس bootloader و kernel و در ادامه بقیه اجزا هش گرفته می شوند. به این ترتیب یک زنجیره اعتماد ایجاد می شود که در آن هر مرحله وضعیت مرحله قبلی را نیز در خود منعکس می کند. اگر حتی بخش کوچکی از یکی از اجزا تغییر کند، مقدار هش متفاوت خواهد شد و در نتیجه مقدار PCR نیز تغییر می کند. بنابراین TPM می تواند تغییرات غیرمجاز مانند وجود bootkit یا rootkit را تشخیص دهد.

ویژگی مهم PCR ها این است که مقدار آن ها مستقیماً overwrite نمی شود، بلکه فقط از طریق عملیاتی به نام extend تغییر می کنند. در عملیات extend، مقدار جدید PCR از ترکیب مقدار قبلی PCR و ورودی جدید به دست می آید.

این طراحی باعث می شود PCR ها در برابر دستکاری مقاوم باشند زیرا مهاجم نمی تواند مقدار دلخواهی را مستقیم داخل PCR قرار دهد. همچنین هر مقدار جدید به مقدار قبلی وابسته می باشد، بنابراین اگر مهاجم بخواهد یکی از مراحل بوت را تغییر دهد، کل زنجیره هش ها تغییر خواهد کرد. در نتیجه بازگرداندن یک مقدار قدیمی یا جعل وضعیت سالم سیستم تقریباً غیرممکن می شود. این خاصیت را Tamper Resistance می نامند.

PCR ها همچنین پایه اصلی Remote Attestation هستند. در Remote Attestation، سیستم مقادیر PCR را از TPM دریافت می کند و TPM این مقادیر را با یک Attestation Key امضا می کند. سپس این اطلاعات به یک سیستم راه دور ارسال می شود. سیستم راه دور امضا را بررسی کرده و مقادیر PCR را با مقادیر مورد انتظار مقایسه می کند. اگر مقادیر با وضعیت سالم و مورد اعتماد سیستم مطابقت داشته باشند، سیستم معتبر شناخته می شود. اما اگر bootloader، kernel یا firmware دستکاری شده باشند، مقادیر PCR متفاوت خواهد بود و فرآیند Attestation شکست می خورد.

قابلیت sealing و unsealing نیز به PCR ها وابسته می باشند. در TPM می توان داده یا کلیدی را به وضعیت خاصی از سیستم وابسته نمود. به این صورت هنگام seal کردن، TPM مقادیر فعلی PCR را همراه داده ذخیره می کند. سپس هنگام unseal کردن، TPM ابتدا PCR های فعلی را بررسی می کند. اگر این مقادیر دقیقاً با مقادیر ذخیره شده مطابقت داشته باشند، TPM داده یا کلید را آزاد می کند. اما اگر وضعیت بوت تغییر کرده باشد و مقادیر PCR متفاوت باشند، عملیات unseal شکست می خورد.

سوال ۳

- Password Authorization: در این روش دسترسی به یک شی یا کلید فقط با دانستن یک مقدار مخفی مانند password یا authorization value امکان پذیر است. این ساده ترین روش احراز هویت در TPM 2.0 محسوب می شود و مشابه استفاده از رمز عبور معمولی است.
 - HMAC Authorization: در این مکانیزم از session و توابع رمزنگاری برای اثبات مالکیت authorization value استفاده می شود، بدون اینکه خود رمز مستقیماً ارسال شود. این روش در برابر حملات شنود و replay attack امنیت بیشتری دارد.
 - Extended Authorization Policy: این مهم ترین و قدرتمندترین مکانیزم TPM 2.0 محسوب می شود. در این روش، دسترسی فقط به دانستن یک password وابسته نیست، بلکه TPM مجموعه ای از شرایط و policy ها را بررسی می کند. این policy ها می توانند شامل مقدار PCR ها، وضعیت Secure Boot، زمان، دستورات مجاز، یا حتی ترکیبی از چند شرط امنیتی باشند.
- مزیت اصلی Extended Authorization Policy نسبت به password ساده این است که دسترسی را به یک وضعیت قابل اعتماد سیستم وابسته می کند. در روش password-based، اگر مهاجم رمز را بداند، می تواند بدون توجه به وضعیت سیستم به داده دسترسی پیدا کند. اما در Extended Authorization Policy حتی اگر مهاجم به فایل ها یا object دسترسی داشته باشد، تا زمانی که شرایط policy برقرار نباشد TPM اجازه استفاده از آن را نمی دهد.
- این موضوع مستقیماً در تمرین sealing و unsealing روی PCR16 که در بخش عملی انجام شد دیده می شود. در این تمرین ابتدا یک policy بر اساس مقدار PCR16 ساخته شد؛ یعنی TPM فقط زمانی اجازه unseal کردن داده را می داد که PCR16 شامل مقدار STATE_HEALTHY باشد.
- در حالت اول، زمانی که PCR16 مقدار صحیح را داشت، عملیات unseal موفق بود و TPM رشته مخفی را برگرداند. دلیل موفقیت این بود که مقدار فعلی PCR دقیقاً با policy تعریف شده مطابقت داشت، بنابراین TPM سیستم را در وضعیت trusted تشخیص داد.
- در حالت دوم، زمانی که PCR16 به مقدار STATE_MALICIOUS تغییر داده شد، عملیات unseal شکست خورد و TPM خطای برگرداند. علت این اتفاق این بود که policy تعریف شده فقط اجازه دسترسی در صورت وجود مقدار STATE_HEALTHY را می داد.
- در حالت سوم، وقتی PCR16 دوباره به مقدار اولیه STATE_HEALTHY بازگردانده شد، عملیات unseal دوباره موفق شد. این نشان می دهد که TPM داده را نه بر اساس password بلکه بر اساس وضعیت اندازه گیری شده و قابل اعتماد سیستم آزاد می کند.